Backend-ul, feature cu feature

Spring Boot 4.1 · Java 21 · MySQL 8 · Docker · GitHub Actions · EC2 · Flavius

Ținta e în `slot-mate-docs/`: `STRUCTURA_BD.pdf` = tabelele, `openapi.yaml` = contractul (îl deschizi în `editor.swagger.io`; DTO-urile = schemele, metodele din controllere = `operationId`).

Fiecare feature trece prin același drum:

migrare `Vn__…sql` → entitate → repository → service → controller → test de integrare → push → verde în Actions → verificat pe EC2

1 Categorii — scheletul, cu tot drumul până pe EC2

a) Fundația proiectului

- Toolchain Java 21; scoate `restdocs` și `asciidoctor`; mută testul în pachetul `ro.mycode.app`.
- `mysql-connector-j` (runtime), `spring-boot-starter-flyway` + `flyway-mysql`; `spring.jpa.hibernate.ddl-auto=validate`.
- Conexiunea din variabile de mediu: `DB_URL`, `DB_USER`, `DB_PASSWORD`, cu valori implicite pentru local.
- Testele pe MySQL din Testcontainers: `spring-boot-testcontainers`, `@ServiceConnection`.

b) Codul

- `V1__categories.sql`: tabelul `categories` + câteva rânduri (Frizerie, Stomatologie, Sală de sport).
- Pachetul `category`: `Category`, `CategoryRepository`, `CategoryService`, `CategoryController`; DTO-uri `CategoryRequest` / `CategoryResponse` ca `record`.
- `getCategories` și `createCategory` exact ca în `openapi.yaml`: nume gol → 400, nume existent → 409.
- Erorile într-un singur `@RestControllerAdvice`, ca `ProblemDetail` — îl refolosești la toate feature-urile.
- Un test de integrare (`@SpringBootTest` + MockMvc): adaugi → apare în listă; duplicat → 409; nume gol → 400.

c) Docker, local

- `slot-mate-api/Dockerfile` în două etape: build cu Gradle → rulare pe `eclipse-temurin:21-jre`.
- `slot-mate-ops/docker-compose.yml`: `mysql` (volum + `healthcheck`) și `api` (`depends_on` cu `condition: service_healthy`, port 8080).
- Parolele vin din `slot-mate-ops/.env`, care e în `.gitignore`; în repo pui doar `.env.example`.

> **Gata când:** `docker compose up --build` → `GET http://localhost:8080/api/categories` = 200.

d) GitHub Actions

- Fișierul stă în `.github/workflows/api.yml`, **la rădăcina repo-ului** (nu în `slot-mate-ops/`). Pornește la push pe `master` când se schimbă `slot-mate-api/**` sau `slot-mate-ops/**`.
- Job `test`: `./gradlew test` (runner-ul `ubuntu-latest` are Docker, deci Testcontainers merge).
- Job `image`: imaginea în GHCR, `ghcr.io/tomescuflavius/slot-mate-api`, cu tag-urile `latest` și SHA-ul commit-ului; platforma = arhitectura instanței EC2 (`amd64` sau `arm64`). Pachetul îl faci public, ca EC2 să-l poată trage fără login.
- Job `deploy`: `scp` cu `docker-compose.yml` pe EC2, apoi prin `ssh`: `docker compose pull && docker compose up -d`. Secrete în repo: `EC2_HOST`, `EC2_USER`, `EC2_SSH_KEY`.

e) EC2

- Ubuntu, Docker + pluginul compose. Security Group: 22 doar de la IP-ul tău, 8080 deschis (temporar, până la nginx).
- Folderul `~/slot-mate/` cu `.env` pus o singură dată, de mână. `.env` nu trece niciodată prin repo.

> **Gata când:** un push pe `master` → workflow verde → `curl http://<ip-ec2>:8080/api/categories` = 200.

2 Conturi

- `V2__users.sql`; termini `User` după `STRUCTURA_BD.pdf`; `register`, `login`, `getCurrentUser`.
- Securitate: `spring-boot-starter-security` + `spring-boot-starter-oauth2-resource-server`, JWT HS256 (`NimbusJwtEncoder` / `Decoder`), claim `role` → `ROLE_…`, parole cu BCrypt, `@EnableMethodSecurity`. Publice: `register`, `login`, GET-urile din contract, Swagger UI, `/error`.
- `JWT_SECRET` în `.env` pe EC2 și ca secret în Actions — niciodată în repo. `createCategory` devine doar pentru `ADMIN`.
- `springdoc-openapi-starter-webmvc-ui` **3.1.x** (2.x nu merge pe Boot 4).

> **Gata când:** pe EC2: `register` → `login` → `/api/users/me` cu `Bearer` = 200; fără token = 401; `POST /api/categories` fără ADMIN = 403.

3 Furnizori

- `V3__providers.sql`; `createMyProvider`, `getMyProvider`, `updateMyProvider`, `getProvider`, `searchProviders` (filtre opționale `categoryId`, `city`, cu `@Query`).
- Un utilizator are cel mult un profil de firmă: al doilea → 409.

> **Gata când:** pe EC2, un furnizor nou apare în `GET /api/providers?city=…`.

4 Servicii

- `V4__offerings.sql`; `createOffering`, `updateOffering`, `deactivateOffering` (`active = false`), `getProviderOfferings` (doar active).
- Proprietarul se verifică în service: serviciul altui furnizor → 403.

> **Gata când:** testul „furnizorul A modifică serviciul lui B" = 403 trece în Actions.

5 Intervale

- `V5__slots.sql`, cu unicitatea pe perechea `(offering_id, start_time)` în `@Table(uniqueConstraints = …)`; `createSlot`, `getSlots` (`from`, `to`, `available`), `deleteSlot`.
- `endTime` = `startTime` + durata serviciului; interval în trecut → 400; același `startTime` → 409.

> **Gata când:** pe EC2, intervalele unei săptămâni apar ordonate după `startTime`.

6 Rezervări

- `V6__bookings.sql` (`slot_id` unic); `createBooking`, `getMyBookings`, `getMyProviderBookings`, `cancelBooking` (șterge rândul).
- Interval deja rezervat → 409, inclusiv când îl prinzi din `DataIntegrityViolationException`.
- Testul de integrare al întregului flux, **prin login real** (nu doar `@WithMockUser`): rezervă (201) → a doua oară (409) → anulează (204) → rezervă din nou (201).

> **Gata când:** fluxul complet merge din Swagger UI-ul de pe EC2, iar acesta are aceleași 21 de operații ca `openapi.yaml`.

> **Spring Boot 4 față de exemplele de pe internet (scrise pe Boot 3):** `@MockBean` → `@MockitoBean` · `@WebMvcTest` din `org.springframework.boot.webmvc.test.autoconfigure` · `@DataJpaTest` din `org.springframework.boot.data.jpa.test.autoconfigure` · Jackson 3: `tools.jackson.databind.ObjectMapper` · starter-ele de test sunt separate: `…-webmvc-test`, `…-data-jpa-test`, `…-validation-test`.